
Modular Programming – Practical Work

Week 5

Classes and Objects, a deeper look 2

Notations: W1Q1 = Week 1, Questions of type 1

Question types:

- Type 1: theory, concepts *knowledge*
- Type 2: understanding typically based on code reading *understanding*
- Type 3: producing yourself code *know-how*

A moodle deposit space is open every week for you to upload your solutions. Solutions of the exercises are provided one week later.

Important points:

- 602_F and 602_D classes should upload solutions **before Tuesday 24h00 the following week**
- **2 or 3 practical work will be corrected** and will count for one third of the intermediate test note

Conventions about fonts:

- Lucida Console is used for Java reserved commands
- Courier is used for the code
- Arial is used for the text

W5Q1 – 1

1. What is the difference between inheritance and composition?

An inheritance is used to implement a relationship "is a". A composition is used to implement a relationship "has a". A subclass that inherits from a superclass can access all members of the superclass that are declared **public** or **protected**. A class that is composed of objects that are instances of other classes can only access **public** members of these other classes (unless those other classes are in the same package, in which case the **protected** members are also available)

2. What is the difference between `final` and `finalize` ?

The `final` keyword is a modifier meaning that a field variable can't be modified anymore. `finalize()` is a method which is called automatically by the Garbage Collector when there are no more references pointing to an object and when this object is going to be cleaned out from the memory

3. What is an implicit cast? What is an explicit cast? In which case an explicit cast will generate an error

Cf D&D v7 p. 150-151, 251(table)

Cf D&D v6 p. 146-147, 242 (table)

In Java, the arithmetic operators work on operands of the same type. An implicit conversion of type (implicit cast) is performed when the operands are of different types and when the rules of promotions are followed (cf table p 242 or 251).

Type	Valid promotions
<code>double</code>	None
<code>float</code>	<code>double</code>
<code>long</code>	<code>float</code> or <code>double</code>
<code>int</code>	<code>long</code> , <code>float</code> or <code>double</code>
<code>char</code>	<code>int</code> , <code>long</code> , <code>float</code> or <code>double</code>
<code>short</code>	<code>int</code> , <code>long</code> , <code>float</code> or <code>double</code> (but not <code>char</code>)
<code>byte</code>	<code>short</code> , <code>int</code> , <code>long</code> , <code>float</code> or <code>double</code> (but not <code>char</code>)
<code>boolean</code>	None (<code>boolean</code> values are not considered to be numbers in Java)

Fig. 6.5 | Promotions allowed for primitive types.

It is also possible to force a conversion of the types (explicit cast). This is used, for example, when we would like to obtain a result of type `float` while the operands are of type `int`.

An explicit cast will generate an error (exception) when the transformation of a type into another one is not possible by the language.

4. Explain the operation of the access modifier protected ?

protected is an access modifier as well as **public** or **private**. **protected** can make a member of a class (variable or method) accessible from any subclass that inherits from the class where the member is defined (and also visible from all classes in the same package).

5. Why and when should we use the instanceof operator? Can we apply the instanceof operator to primitive types ?

instanceof is an operator that returns a boolean value. It takes as input an object (to the left of instanceof) and a class name (to the right of instanceof). It returns true if the object is of the type specified by the class, else it returns false. The use of instanceof allows to avoid mistakes bound to inappropriate castings.

W5Q2 – 1 IMPLICIT CAST

Have a look at the following code :

```
int number_1 = 7;
int number_2 = 3;
float number_3 = 7;
float number_4 = 3;
int result_1 = 0;
float result_2 = 0;

result_1 = number_1 / number_2;
result_2 = number_1 / number_2;
result_1 = number_3 / number_2;
result_2 = number_3 / number_2;
result_1 = number_1 / number_4;
result_2 = number_1 / number_4;
result_2 = number_3 / number_4;
```

- Which of these statements do not compile? Why? What can be done to resolve the problem (to compile the program)?

result_1 = number_3 / number_2;
result_1 = number_1 / number_4;

Type mismatch: cannot convert from float to int => float result_1

- What are the results of the above calculations? Which calculations do not give the result 2.3333333? Why?

```

2          int / int -> digits lost, -> int (result)
2.0        int / int -> digits lost
           -> implicit typecast float-> 2.0
2          float / int -> no digits lost
           -> int; digits lost here
2.3333333  float / int -> no digits lost
           -> float; no digits lost
2          int / float -> no digits lost
           -> int; digits lost here
2.3333333  int / float -> no digits lost
           -> float; no digits lost
2          float / float -> no digits lost
           -> implicit typecast int -> 2
2.3333333  float / float -> no digits lost
           -> float; no digits lost

```

- If you substitute the digits 7 and 3 for the variables in statement two, which three different ways are there to change this statement so that it results in the correct value?

Three Options:

- A) add ".0" to one number -> read as double (result has to be of type double)
- B) add "f" on one of the numbers -> read as float
- C) use an explicit typecast (float)

```

int number_1 = 7.0;
int number_2 = 3f;

```

```

result_2 = number_1 / number_2;
result_2 = number_1 / number_2;
result_2 = (float) number_1 / number_2;

```

W5Q2 – 2 By value / by reference

Have a look at the following code :

```
class MyInteger
{
    private int value;
    public MyInteger( int value ) { this.value = value; }
    public void setValue( int n ) { this.value=n; }
    public int getValue() { return this.value; }
    public String toString() { return "" + value; }
}

public class Test {

    public static void main(String[] args) {

        // Pt.1. passing by value
        int a = 2;
        changeSimpleType( a );
        System.out.println( "a = " + a );

        // Pt.2. passing by reference
        Integer b = new Integer( 2 );
        changeIntegerType( b );
        System.out.println( "b = " + b );

        // Pt.3. passing by reference (bis)
        MyInteger c = new MyInteger(2);
        changeMyIntegerType( c );
        System.out.println( "c = " + c );

    }

    public static void changeSimpleType( int a )
    {
        a = 10;
    }

    public static void changeIntegerType( Integer a )
    {
        a = 10;
    }

    public static void changeMyIntegerType( MyInteger a )
    {
        a.setValue( 10 );
    }
}
```

Question 1: What is the value of variable `a` after a call to the method `changeSimpleType()` ?

a equals 2 -> passing by value

Question 2: What is the value of variable `b` after a call to the method `changeIntegerType()` ?

b equals 2 -> passing by reference

The assignment of an int value in an Integer with 'a=10' is re-written by the compiler as 'a = new Integer(10)'. The effect of this re-writing is a loss of the reference previously stored in a. The Integer object passed as parameter of the method is then unchanged. The automatic rewriting of int values into Integer object instantiation is called "autoboxing" (Google --> autoboxing java int Integer).

Question 3: What is the value of variable `a` after a call to the method `changeMyIntegerType()` ?

c equals 10 -> passing by reference

Question 4: When you run this code, what do you remark? Why does the value of `b` remain unchanged even if it is passed as a reference? Check the methods of the `MyInteger` and the `Integer` classes to help you find the answer.

Cf. question 2

W5Q2 – 3 This

What is the output of the program `TestCircle` below? (answer without copying the code in Eclipse).

Note: Be careful using of `this` because here it is automatically transformed into a call to `toString()` which is redefining itself in the superclass and subclass.

```
/**
 * Point class represents a basic x-y coordinate pair
 *
 * @author jean.hennebert@hevs.ch
 */
public class Point {

    private int x; //x coordinate
    private int y; //y coordinate

    //no argument constructors
    public Point() {
        x = 0;
        y = 0;
        System.out.println("Point constructor (no argument): " + this);
    }

    public Point(int xValue, int yValue) {
        x = xValue;
        y = yValue;
        System.out.println("Point constructor: " + this);
    }

    public int getX() {
        return x;
    }

    public void setX(int x) {
        this.x = x;
    }

    public int getY() {
        return y;
    }

    public void setY(int y) {
        this.y = y;
    }

    public String toString() {
        return "[" + getX() + "," + getY() + "]";
    }
}
```

```

/**
 * Circle class declaration
 * @author jean.hennebert@hevs.ch
 *
 */
public class Circle extends Point {

    private double radius;

    public Circle() {
        setRadius(0.0);
        System.out.println("Circle constructor (no arguments): " + this);
    }

    public Circle(int xValue, int yValue, double r) {
        super(xValue, yValue);
        setRadius(r);
        System.out.println("Circle constructor: " + this);
    }

    public double getRadius() {
        return radius;
    }

    public void setRadius(double r) {
        if(r < 0.0) radius = 0;
        else radius = r;
    }

    public String toString() {
        return "center=" + super.toString() + " radius=" + getRadius();
    }
}

public class TestCircle {

    /**
     * @param args
     */
    public static void main(String[] args) {

        Point p = new Point(5,10);
        Circle c2 = new Circle(p.getX(),p.getY(),2.0);

        Circle c1 = new Circle();

    }
}

```



```
Point constructor: [5,10]
Point constructor: center=[5,10] radius=0.0
Circle constructor: center=[5,10] radius=2.0
Point constructor (no argument): center=[0,0] radius=0.0
Circle constructor (no arguments): center=[0,0] radius=0.0
```

Commentary: Beware the use of `this` in the constructor of the superclass `Point` that will automatically call `toString()` when it is cast to a `String`. If the object that corresponds to `this` is of type `Circle`, then it is the method `toString()` from `Circle` that is called (and not that of `Point`), where the emergence of `radius = 0.0` in the second line of the program output.

CORRECTION